

ISEL | DEI | AMD / MLDM

Machine Learning and Data Mining

1. The MoP goal and general information (*just read; do nothing yet!*)

This Module-of-Practice (MoP) prepares the working-environment (to be used in MLDM course) via provided python scripts.

Important: the **MoP folders** must **always be placed inside** the course workspace folder,

- **named** `mLDm_MoP_workspace`

At the end of this MoP, each student should have:

- the common MLDM workspace structure created
- a python virtual environment created in `mLDm_MoP_workspace/___venv/`
- the required python packages installed
- the *PostgreSQL* container configuration created
- the environment checked through simple python scripts
- generated evidence saved in `03_output/` and `04_report/`

1.1. The common-folder structure for the course is:

```
mLDm_MoP_workspace/  
  __container/  
    postgres/  
      compose.yaml  
      data_pgadmin/  
      data_postgres/  
  __venv/  
  MoP_NN_<description>/  
    _handout.pdf  
    01_input/  
    02_code/  
    03_output/  
    04_report/
```

Notice that each MoP has its own folder, `MoP_NN_<description>`, where:

- `NN`, is the MoP number, e.g., 01, 02, 03, ..., and
- `<description>`, is a short description written in `snake_case`.

For example, the name of the folder for this MoP is `MoP_01_install_and_test`.

1.2. The name and role of each folder (or file) in `mLDm_MoP_workspace/` is:

- `__container/` stores configuration and persistent data of container services.
- `postgres/` stores *PostgreSQL* service configuration and persistent data.
- `compose.yaml` defines the *PostgreSQL* and *pgAdmin* container services.

- `data_pgadmin/` stores persistent data used by pgAdmin.
- `data_postgres/` stores persistent data used by *PostgreSQL*.
- `__venv/` stores the python virtual environment used in the course.

1.3. The name and role of each MoP-specific folder is:

- `MoP_NN_<description>/` the working folder for one specific MoP
- `_handout.pdf` contains the instructions for the MoP
- `01_input/` contains the input files provided or used in the MoP
- `02_code/` contains scripts, notebooks, or other code files used in the MoP
- `03_output/` contains files generated by scripts or notebooks
- `04_report/` contains answers, evidence, or reports produced by the student

Important:

- contents of `03_output/` should always be reproducible from `01_input/` and `02_code/`.

1.4. For this MoP the `02_code/` scripts (that create and test) the environment are:

- `requirements.txt`, lists the python packages to install in the virtual environment
- `create_workspace.py`, creates or completes the common workspace structure
- `create_venv.py`, creates the python virtual environment
- `create_compose.py`, creates the *PostgreSQL* container configuration file
- `check_environment.py`, runs simple checks and writes evidence to `03_output/`
- `build_submission_zip.py`, builds the "zip" file to submit

1.5. (very) Important name convention to be adopted:

- the files provided by the course use the prefix `_mLDm_`
- these support files **are not be renamed or edited** unless explicitly requested
- files created or modified by students should use the same base name without the `_mLDm_`

For example:

- `_mLDm_02_create_venv.py` is a file provided by the course,
- while `02_create_venv.py` could be a student-created or student-modified version.

2. Build the environment (*now: read and understand, then do!*)

Recall: provided scripts are located in `MoP_01_install_and_test/02_code/`.

2.1. | step-1 | create the workspace structure

Before proceeding make sure that this MoP folder is inside:

- `mLDm_MoP_workspace/`

Therefore, **if the folder** `mLDm_MoP_workspace/` **does not yet exists, create it.**

On Unix-like, from the folder that contains `MoP_01_install_and_test/`, run:

- `mkdir -p mLDm_MoP_workspace`

Then move the MoP folder into the workspace:

- `mv MoP_01_install_and_test mLDm_MoP_workspace/`

After that, check that the folder structure is:

```
mLDm_MoP_workspace/  
  MoP_01_install_and_test/  
    01_input/  
    02_code/  
    03_output/  
    04_report/
```

Now, analyze and then execute `_mLDm_01_create_workspace.py`; the script does not need to be executed from inside the `02_code` folder; it uses its own file location to infer the intended structure. So, if terminal is open in `MoP_01_install_and_test/02_code/`, run:

- `python3 _mLDm_01_create_workspace.py`

On some systems, e.g., MS-Windows, you may need to use:

- `python _mLDm_01_create_workspace.py`

or:

- `py _mLDm_01_create_workspace.py`

The expected result is that `mLDm_MoP_workspace/` contains both the current MoP folder and the common course folders:

```
mLDm_MoP_workspace/  
  __container/  
    postgres/  
      compose.yaml  
      data_pgadmin/  
      data_postgres/  
  __venv/  
  MoP_01_install_and_test/  
    01_input/  
    02_code/  
    03_output/  
    04_report/
```

If `mLDm_MoP_workspace` already exists, the script does not change the existing structure and return a message similar to:

```
PTS | mLDm | > workspace-ready: <your-path>/mLDm_MoP_workspace
```

2.2. | step-2 | create and test the python virtual environment

The virtual environment, or `venv`, is used to install the python packages required for the course in an isolated environment. Although these packages could be installed directly on the machine, this MoP adopts the `venv` approach to make the setup cleaner and more reproducible.

Other approaches may be used, as long as the final environment works properly.

Analyze and then execute (in a similar way as in previous step):

```
_mLDm_02_create_venv.py
```

The python virtual environment is created in:

```
mLDm_MoP_workspace/___venv/
```

The packages and its usage (cf., `_mLDm_00_requirements.txt`):

- `numpy`, supports numerical computation
- `pandas`, supports tabular data manipulation
- `duckdb`, supports local SQL-based analytical processing
- `psycopg[binary]`, supports python connections to *PostgreSQL*
- `scikit-learn`, supports machine learning models, pipelines, and evaluation
- `matplotlib`, supports standard static plots
- `plotly`, supports interactive visualisation
- `jupyterlab`, provides the notebook environment
- `ipykernel`, allows the virtual environment to be used as a Jupyter kernel
- `ipywidgets`, supports interactive elements in notebooks

Now, let's test the venv:

- first, activate it, so **move to the** `mLDm_MoP_workspace` **folder** and run the command:

```
source ___venv/bin/activate (on macOS or Linux), or  
___venv/Scripts/activate.bat (on MS-Windows Command-Prompt), or  
___venv/Scripts/Activate.ps1 (on MS-Windows PowerShell)
```

- then, run the following tests:

```
python --version
```

```
python -m pip --version
```

```
python -m pip list
```

the next test may take some additional time to execute:

```
python -c "import numpy, pandas, duckdb, psycopg, sklearn, matplotlib,  
plotly, jupyterlab, ipykernel, ipywidgets; print('python virtual envi  
ronment OK')"
```

- finally, at the end, deactivate the venv:

```
deactivate
```

2.3. | step-3 | install (or confirm) the container tool

This MoP uses container services to run *PostgreSQL* and *pgAdmin*. For that, a container tool must be installed. Here, the supported and assumed tool is *podman* because it is a free and open-source container tool. The *Podman-Desktop* also provides a graphical interface, which helps to inspect containers, images, volumes, logs, and running services.

However, if you have docker installed and working, you may use it instead. In that case, you do not need to install podman, but you must adapt the container commands accordingly.

So, use one of the following options:

- install and use podman, preferably with Podman-Desktop
- use an already installed and working docker environment

To check whether podman or docker is already available, run:

```
podman --version or docker --version
```

At least one of these commands should work. If neither podman nor docker is available, install Podman-Desktop for your operating system. After installation, open Podman-Desktop and complete the initial setup for container workloads.

On macOS and MS-Windows, Podman usually requires a Podman machine, i.e., a small Linux virtual machine used to run the container engine. If Podman-Desktop asks you to create or start a Podman-machine, accept the default option.

Also make sure that the Compose support is available. In Podman-Desktop, this may be configured through:

- Settings > Resources > Compose

After configuring Podman-Desktop, check that the command-line tools are available:

```
podman --version
```

and:

```
podman compose version
```

If you decide to use docker instead, check:

```
docker --version and docker compose version
```

Important: the following steps in this MoP assume podman. If you use docker, replace commands starting with podman by the equivalent commands starting with docker.

For example, to start the services:

```
podman compose up -d becomes docker compose up -d
```

To list the running containers:

```
podman ps becomes docker ps
```

To stop and remove the services created by the compose configuration, open a terminal in:

```
mLDm_MoP_workspace/__container/postgres/
```

and run:

```
podman compose down or docker compose down
```

Before continuing, make sure that either podman or docker is installed, that the corresponding compose command is available, and that the graphical tool can be opened.

2.4. | step-4 | create the container configuration

The `compose.yaml` file defines the container services used in the course. For this MoP, the expected services are:

- *PostgreSQL*
- `pgAdmin`

Analyze and then execute (in a similar way as in previous step):

`_mLDm_03_create_compose.py`

This script creates the file (and does not overwrite if already exists):

`mLDm_MoP_workspace/___container/postgres/compose.yaml`

2.5. | step-5 | start the container services

After `compose.yaml` has been created, open a terminal in:

`mLDm_MoP_workspace/___container/postgres/`

Start the services with:

```
podman compose up -d
```

Check that the services are running:

```
podman ps
```

Important: if you use `docker`, replace commands starting with `podman` by `docker`.

3. Check the environment

Analyze and then execute:

`_mLDm_04_check_environment.py`

This script check (at least):

- the `python` version
- the active virtual environment
- the installed `python` packages
- the ability to import the required packages
- the availability of `JupyterLab`
- the availability of `Podman` or `Docker`
- the status of the *PostgreSQL* container
- the ability to connect to *PostgreSQL* from `python`

The script write evidence files to:

`MoP_01_install_and_test/03_output/`

Check the evidence files:

- `01_check_python.txt`
- `02_check_package.txt`

- 03_check_container.txt
 - 04_check_postgres.txt
-

4. Test pgAdmin in the browser

After checking the environment, also test pgAdmin interactively through the browser.

First, make sure that the container services are running:

```
podman ps
```

And, if not running, start them (`podman compose up -d`).

Then, open a browser and access:

`http://localhost:5050`

login with (cf., `compose.yaml`),

- email: `admin@example.com`
- password: `admin`

After entering pgAdmin, register the *PostgreSQL* server:

- Servers <right-button> Register > Server...

Use a name such as:

- `mldm_postgres`

Select the <Connection> fold and use:

- host name/address: `postgres`
- port: `5432`
- maintenance database: `mldm`
- username: `mldm`
- password: `mldm`

Notice that `admin` is only used to log in to pgAdmin; the *PostgreSQL* database user used in this MoP is `mldm`.

Important: inside pgAdmin, the *PostgreSQL* host is `postgres`, not `localhost`; this happens because pgAdmin and *PostgreSQL* are running as services inside the same *Compose* configuration.

After registering the server, check that you can expand the server tree and see the database:

- `mldm`

This confirms that:

- the pgAdmin web interface is available
- the *PostgreSQL* container is running
- pgAdmin can connect to *PostgreSQL*
- the database configuration is correct

If the browser cannot open `http://localhost:5050`, check again whether the containers are running and whether the container services were started from:

```
mLDm_MoP_workspace/___container/postgres/
```

If needed, to stop the services, remove data and restart the services:

```
podman compose down
rm -rf data_pgadmin
mkdir data_pgadmin
podman compose up -d
```

Notice that `rm` and `mkdir` are Unix commands; use corresponding for your operating system.

5. Test the notebook environment

The python virtual environment can also be used from a Jupyter notebook. This is important because some MoP may use scripts, while others may use a notebook. Therefore, the same environment should support both execution modes.

Activate the virtual environment from the `mLDm_MoP_workspace` folder:

```
source ___venv/bin/activate
```

If on MS-Windows Command-Prompt or PowerShell, use:

```
___venv/Scripts/activate.bat, or
___venv/Scripts/Activate.ps1
```

Then, start JupyterLab:

```
jupyter lab
```

A browser window should open. If it does not open automatically, copy the URL shown in the terminal and paste it in the browser.

Create a new notebook and save it as:

```
MoP_01_install_and_test/04_report/01_my_notebook_check.ipynb
```

5.1. Execute test-cell (provided) code

In the first notebook cell, write and execute:

```
import sys
print( "Jupyter notebook OK" )
print( sys.executable )
```

Check that the path printed by `sys.executable` refers to the virtual environment:

```
mLDm_MoP_workspace/___venv/
```

In a second notebook cell, write and execute:

```
import numpy
import pandas
import duckdb
import psycopg
```

```
import sklearn
import matplotlib
import plotly
import jupyterlab
import ipykernel
import ipywidgets
print("Required packages imported successfully")
```

In a third notebook cell, test a simple widget:

```
import ipywidgets as widgets
widgets.IntSlider(
    value=5,
    min=0,
    max=10,
    description="test" )
```

The slider should be displayed below the cell.

5.2. Build your-own test-cell by completing (the provided) code

For this task, use the template file in:

```
MoP_01_install_and_test/
  01_input/
    _mLDm_notebook_evidence_to-complete.txt
```

Open the file, copy its content into a new notebook cell, and complete the placeholders directly in the notebook.

Important: do **not** edit the original `_mLDm_notebook_evidence_to-complete.txt` file; it is an input/template file provided by the course (*recall* the `_mLDm_` prefix rule).

The validation file must be created in:

```
MoP_01_install_and_test/
  04_report/
    01_my_notebook_check.txt
```

After executing the cell, open the generated file and check that it contains information about:

- whether the notebook kernel uses `mLDm_MoP_workspace/__venv/`
- whether the virtual environment required packages were detected
- any problem found during the validation

If the line about the virtual environment says no, check whether the notebook is using the correct kernel.

Recall: the overall "pipeline" is:

```
01_input/_mLDm_notebook_evidence_to-complete.txt
| copy to notebook cell
| complete the placeholders
| execute the cell
| generate 04_report/01_my_notebook_check.txt
```

At this point, the notebook should contain both:

- the executed validation cell, and
 - the output produced by that cell
-

5.3. Done with *JupyterLab*

Finally, stop JupyterLab from the terminal where it is running, usually with:

- CTRL - C

and, when asked, confirm with 'y'.

At the end of this section, you should have, in the `04_report/` folder:

- a notebook saved in `01_my_notebook_check.ipynb`
 - a notebook test evidence saved in `01_my_notebook_check.txt`
-

6. Evidence to submit

The folder `03_output/` should contain the evidence files generated by `_mLDM_04_check_environment.py`:

- `01_check_python.txt`
- `02_check_package.txt`
- `03_check_container.txt`
- `04_check_postgres.txt`

These files provide evidence that:

- the python virtual environment exists and can be used
- the required packages are installed and can be imported
- at least one container tool, podman or docker, is available
- the container services can be inspected
- *PostgreSQL* can be accessed from python

The folder `04_report/` should contain the notebook evidence produced by the student:

- `01_my_notebook_check.ipynb`
- `01_my_notebook_check.txt`

These files provide evidence that:

- JupyterLab opened successfully
- a notebook was created and executed
- the notebook uses the expected python environment
- the required packages can be imported from the notebook
- the notebook can generate and save a validation output

No additional written report is required for this MoP. However, if any check fails, you should add a separate file named:

- 04_report/problems_work_in_progress.txt

The file should briefly describe:

- what failed
 - what was tried
 - what problems the student still needs to fix
-

7. Build the submission "zip" file (with the provided submission tool)

At the end of each MoP, create the "zip" file to submit in the Moodle platform. The "zip" file only contains:

- 03_output/
- 04_report/

The "zip" file name includes the group number and the MoP folder name. For example, for group 3 and MoP_01_install_and_test, the generated file will be:

- submitted_G03_MoP_01_install_and_test.zip

The submission tool is provided in the MoP 02_code/ folder, as:

- _mLDm_xx_build_submission_zip.py

So, if this is the first time you use the script, run it from the current MoP 02_code/ folder:

```
cd mLDm_MoP_workspace/MoP_NN_<description>/02_code
python3 _mLDm_xx_build_submission_zip.py <group-number>
```

On some systems, e.g., MS-Windows, you may need to use:

```
python _mLDm_xx_build_submission_zip.py <group-number>
```

or:

```
py _mLDm_xx_build_submission_zip.py <group-number>
```

Important: the first time it runs, the **submission tool also copies itself** to the root of:

- mLDm_MoP_workspace/

After that, you may also run it from the workspace folder by explicitly indicating both the group number and the MoP folder name:

```
cd mLDm_MoP_workspace
python3 _mLDm_xx_build_submission_zip.py
      <group-number> MoP_NN_<description>
```

Important: this makes the same submission tool available for any future MoP.

At the end, check that the "zip" file was created inside the mLDm_MoP_workspace, for example, for group number 3:

```
mLDm_MoP_workspace/submitted_G03_MoP_NN_<description>.zip
```

Finally, submit this "zip" file in the Moodle platform.